



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

APLICAȚIE PENTRU REPARTIZAREA LOCURILOR LA CĂMIN

Absolvent

Andrei Timotei

Coordonator științific

Conf.dr. Radu Boriga

București, iunie 2025

Rezumat

Repartizarea locurilor de cazare în căminele universitare este, în mod tradițional, un proces complex, care implică gestionarea manuală a unui volum considerabil de date și preferințe ale studenților. Acest mod de lucru poate fi inefficient și predispus la erori.

Lucrarea de față propune o aplicație care automatizează o mare parte din etapele acestui proces, precum prelucrarea datelor, sortarea studenților și generarea listelor de repartizare. În același timp, sistemul păstrează un grad ridicat de flexibilitate, permițând comisiei de cazare să intervină facil asupra datelor sau rezultatului alocărilor, în funcție de necesități punctuale.

Abstract

The allocation of university dormitory places is traditionally a complex process involving the manual handling of large volumes of data and student preferences. This approach can be inefficient and prone to errors.

This paper proposes an application that automates a significant part of this process, including data processing, student sorting, and the generation of allocation lists. At the same time, the system maintains a high degree of flexibility, allowing the accommodation committee to easily intervene and make adjustments to the data or allocation results as needed.

Cuprins

1	Introducere	5
1.1	Contextul și motivația	5
1.2	Scopul lucrării	5
1.3	Obiective	6
1.4	Structura lucrării	7
2	Preliminarii	8
2.1	Problema practică abordată	8
2.2	Lipsa soluțiilor existente	8
3	Arhitectura	9
3.1	Alegerea tehnologiilor	9
3.1.1	Platforma de dezvoltare	9
3.1.2	Limbaajul de programare	9
3.1.3	Structura sistemului	9
3.1.4	Servicii	10
3.1.5	Modele de date	10
3.2	Tecnologii și biblioteci utilizate	10
3.2.1	Interfață grafică	10
3.2.2	Procesare date	10
3.2.3	Securitate	11
3.2.4	Inteligență artificială	11
3.3	Fluxul de date	11
3.4	Considerații de securitate	11
3.5	Scalabilitate și mentenanță	11
4	Implementare	12
4.1	Gestionarea utilizatorilor și a setărilor	12
4.2	Structura generală și fluxul de date	12
4.3	Gestionarea studenților	13
4.4	Gestionarea căminelor	14

4.5	Algoritmul de repartizare	14
4.5.1	Modul standard (ordonare descrescătoare după punctaj)	14
4.5.2	Modul de satisfacție mutuală (bipartite matching)	15
4.5.3	Etapă de completare (plasare forțată)	15
4.6	Sistemul de recomandare AI	15
4.7	Exportul și prezentarea rezultatelor	16
5	Manual de utilizare	17
5.1	Prezentarea generală a interfeței	17
5.2	Procesarea datelor studenților	18
5.3	Gestionarea resurselor și algoritmul de repartizare	18
5.4	Vizualizarea rezultatelor și statisticilor	19
6	Concluzii	20
6.1	Rezumatul contribuțiilor	20
6.2	Limitări și provocări întâmpinate	20
6.3	Direcții de dezvoltare viitoare	21
6.4	Reflecție personală	21
	Bibliografie	22

Capitolul 1

Introducere

1.1 Contextul și motivația

Gestionarea repartizării locurilor de cazare în căminele universitare este o activitate recurentă, consumatoare de timp, care presupune colectarea și procesarea manuală a unui volum mare de date: cereri ale studenților, medii, situații sociale și preferințe. În mod tradițional, această activitate este realizată de o comisie desemnată, care centralizează informațiile și efectuează alocarea locurilor conform unei metodologii prestabilite a facultății.

Având oportunitatea de a face parte din această comisie, am experimentat direct dificultățile implicate de procesul manual de prelucrare a datelor. Aceste neajunsuri, precum volumul ridicat de muncă repetitivă, lipsa unui control centralizat al situației și potențialele erori de operare, m-au determinat să reflectez asupra unei soluții informatizate care să sprijine desfășurarea acestui proces.

Astfel a apărut ideea dezvoltării unei aplicații care să automatizeze în totalitate aceste operațiuni și să ofere un cadru unitar pentru gestionarea întregului proces. Aplicația este construită special pentru nevoile Facultății de Matematică și Informatică, urmând metodologia proprie de alocare și asigurând totodată flexibilitate pentru intervenții manuale din partea comisiei de cazare.

1.2 Scopul lucrării

Lucrarea urmărește proiectarea și realizarea unei aplicații desktop care să automatizeze procesul de repartizare a locurilor de cazare pentru studenții Facultății de Matematică și Informatică. Scopul este acela de a înlocui procesele manuale repetitive cu un sistem eficient, adaptat metodologiei interne de alocare a locurilor în cămin.

Aplicația oferă atât o variantă clasică de alocare pe baza punctajului, cât și un mod avansat care ține cont de cerințele mutuale ale studenților privind colegii de cameră, opti-

mizând satisfacția acestora. În plus, pentru situațiile în care nu sunt exprimate preferințe explicite, este integrat un sistem de recomandare automată, care utilizează modele de procesare a limbajului natural pentru a sugera cel mai potrivit cămin.

1.3 Obiective

Pentru atingerea scopului propus, lucrarea urmărește îndeplinirea următoarelor obiective:

- Preluarea și prelucrarea datelor din surse multiple:
 - formularele de înscriere completate de studenți;
 - informațiile privind căminele și camerele disponibile, furnizate de direcția de Cămine;
 - datele academice extrase din sistemul UMS al facultății.
- Repartizarea locurilor de cămin în funcție de an de studiu și specializare, conform metodologiei interne.
- Automatizarea procesului de alocare a studenților, luând în considerare punctajul, preferințele pentru cămin și cererile mutuale de colegi de cameră.
- Implementarea unui algoritm alternativ de repartizare, care optimizează satisfacția mutuală a studenților ce doresc să fie plasați împreună.
- Integrarea unui sistem de recomandare bazat pe inteligență artificială, pentru sugestii personalizate în cazul studenților care nu își exprimă clar preferințele.
- Oferirea unei interfețe grafice intuitive, ușor de utilizat de către membrii comisiei de cazare, fără necesitatea unei pregătiri tehnice prealabile.
- Permitea ajustării manuale a rezultatelor generate, pentru a răspunde eventualelor situații particulare.
- Generarea rezultatelor finale în formate standardizate (PDF, Excel), ușor de utilizat în scopuri administrative.
- Asigurarea confidențialității și integrității datelor printr-un mecanism de înregistrare și criptare.

1.4 Structura lucrării

Lucrarea este structurată în următoarele capitole:

1. **Introducere:** conturează contextul, scopul, motivația și obiectivele urmărite.
2. **Preliminarii:** oferă o analiză a contextului instituțional, descrie problema practică abordată, modul în care aplicația acoperă nevoile concrete ale facultății și introduce conceptele-cheie utilizate.
3. **Arhitectura:** prezintă concepția aplicației, componentele majore și raționamentele din spatele deciziilor de proiectare.
4. **Implementare:** detaliază aspectele tehnice ale realizării aplicației, algoritmi folosiți pentru alocare și structurile de date utilizate.
5. **Manual de utilizare:** explică modul de utilizare a aplicației prin exemple, capturi de ecran și scenarii de lucru.
6. **Concluzii:** rezumă contribuțiile lucrării și propune direcții viitoare de îmbunătățire a aplicației.

Capitolul 2

Preliminarii

2.1 Problema practică abordată

Facultatea de Matematică și Informatică organizează anual procesul de repartizare a studenților în cămine, în baza unei metodologii clare care ține cont de media academică, domiciliu, situații sociale și alte criterii. Procesul implică colectarea datelor prin formulare, sortarea cererilor și întocmirea listelor finale.

În prezent, toate aceste activități se realizează manual, utilizând documente Excel, ceea ce face procesul predispus la erori și consumator de timp.

2.2 Lipsa soluțiilor existente

În urma unei analize de piață, nu a fost identificată o aplicație software care să răspundă în mod direct nevoilor specifice ale facultății. Majoritatea aplicațiilor disponibile fie sunt orientate către managementul general al căminelor, fie presupun integrare cu alte platforme universitare, fie sunt specifice metodologiei unei anumite facultăți, fără detalii amănunțite privind algoritmi folosiți în procesul de repartizare și plasare al studenților.

Aplicația propusă vine să suplinească această lipsă, fiind adaptată metodologiei concrete aplicate în cadrul facultății și având ca obiectiv principal ușurința în utilizare și flexibilitatea în operare.

O astfel de soluție trebuie să integreze atât particularitățile metodologice ale procesului de repartizare, cât și nevoia de control, flexibilitate și simplitate în utilizare. În capitolele următoare sunt prezentate arhitectura, modul de implementare, cât și manualul de utilizare al acestei aplicații.

Capitolul 3

Arhitectura

3.1 Alegerea tehnologiilor

3.1.1 Platforma de dezvoltare

Aplicația a fost dezvoltată ca o soluție desktop, o alegere strategică bazată pe considerente de securitate și control. Datele studenților, fiind de natură sensibilă, necesită un nivel ridicat de protecție care este mai ușor de asigurat într-un mediu desktop controlat. Această abordare elimină riscurile asociate cu expunerea datelor pe internet și permite un control strict al accesului, aplicația fiind instalată doar pe calculatoarele autorizate ale comisiei de cazare. De asemenea, procesarea locală oferă avantaje semnificative în ceea ce privește performanța, eliminând latența asociată cu aplicațiile web și permițând o manipulare mai rapidă a volumului mare de date.

3.1.2 Limbajul de programare

Python a fost ales ca limbaj principal de dezvoltare, o decizie motivată de ecosistemul său bogat și versatil. Existența unor biblioteci specializate pentru manipularea datelor, precum `pandas` [3] și `numpy` [2], precum și a unor soluții robuste pentru lucrul cu fișierelor Excel (`openpyxl`), a fost un factor decisiv în această alegere. Sintaxa simplă și clară a limbajului permite dezvoltarea rapidă și mentenanță ușoară, în timp ce comunitatea activă oferă un suport vast și multe resurse pentru rezolvarea problemelor.

3.1.3 Structura sistemului

Aplicația urmează o arhitectură modulară, organizată în componente distincte care îndeplinesc funcții specifice. Această organizare facilitează dezvoltarea, testarea și mentenanța sistemului. Componentele principale includ serviciile, care implementează logica de procesare și prelucrare a datelor, modelele care definesc structurile de date, interfața care gestionează prezentarea și interacțiunea cu utilizatorul.

3.1.4 Servicii

Sistemul este organizat într-o suită de servicii specializate, fiecare având responsabilități bine definite. Există un serviciu dedicat algoritmului de repartizare a studenților pe ani și specializări, care implementează logica complexă de optimizare și alocare. Un alt serviciu se ocupă de gestionarea și organizarea datelor, asigurând stocarea și recuperarea eficientă a informațiilor. Securitatea sistemului este asigurată printr-un serviciu specializat în autentificare și controlul accesului. Procesul de repartizare este coordonat de un serviciu dedicat, care orchestrează întregul flux de lucru. Sortarea și clasificarea datelor sunt gestionate de un serviciu special, în timp ce configurațiile sistemului sunt administrate printr-un serviciu separat, asigurând flexibilitatea întregului sistem.

3.1.5 Modele de date

Structurile de date definește modelează structura cererilor studenților, capturând toate informațiile relevante necesare pentru procesul de repartizare. Aceste structuri includ detalii despre situația academică, preferințele de cazare și alte criterii relevante. De asemenea, există structuri dedicate pentru definirea caracteristicilor căminelor, inclusiv capacitatea, facilitățile disponibile și restricțiile specifice. Prin intermediul acestor structuri de date personalizate, aplicația menține un context permanent asupra datelor și monitorizează în timp real modificările din fișierele de intrare, asigurând consistența și actualizarea automată a informațiilor.

3.2 Tehnologii și biblioteci utilizate

3.2.1 Interfață grafică

Interfața utilizator a fost construită folosind `customtkinter` [7], un framework modern care oferă un aspect profesional și consistent. Această alegere permite crearea unei interfețe intuitive, esențială pentru o experiență de utilizare plăcută. Pentru procesarea și manipularea imaginilor, s-a utilizat biblioteca Pillow, care oferă funcționalități avansate de procesare a imaginilor.

3.2.2 Procesare date

Manipularea eficientă a datelor este asigurată prin utilizarea unui set de biblioteci specializate. Pandas oferă structuri de date puternice pentru manipularea datelor tabulare, în timp ce numpy oferă suport pentru operații numerice și vectoriale complexe. Pentru integrarea cu sistemele existente, openpyxl oferă funcționalități robuste pentru manipularea fișierelor Excel.

3.2.3 Securitate

Securitatea aplicației este asigurată prin implementarea unor mecanisme multiple. Bcrypt este utilizat pentru hashing-ul securizat al parolelor, în timp ce biblioteca cryptography oferă funcționalități avansate pentru operații criptografice. Aceste tehnologii asigură protecția datelor sensibile și integritatea sistemului.

3.2.4 Inteligență artificială

Sistemul de recomandare integrat utilizează tehnologii avansate de inteligență artificială. `Sentence-transformers` [6] oferă capabilități de procesare a limbajului natural, esențiale pentru înțelegerea preferințelor studenților. Framework-ul `PyTorch` [4] oferă suport pentru implementarea modelelor de deep learning.

3.3 Fluxul de date

Procesul de gestionare a datelor în aplicație urmează un flux bine definit, începând cu importul datelor din formularele studenților și sistemul UMS. Datele sunt apoi prelucrate și validate, urmând aplicarea algoritmilor de repartizare. Rezultatele sunt generate în formate standardizate și exportate pentru utilizare administrativă, asigurând o integrare fluidă cu procesele existente.

3.4 Considerații de securitate

Arhitectura aplicației integrează multiple niveluri de securitate pentru a proteja datele sensibile. Sistemul implementează un mecanism robust de autentificare și autorizare, asigurând că doar utilizatorii autorizați au acces la funcționalitățile aplicației. Datele sensibile sunt criptate, iar toate fișierele de intrare sunt validate pentru a asigura formatul corect al datelor.

3.5 Scalabilitate și mentenanță

Arhitectura modulară a aplicației oferă o bază solidă pentru scalabilitate și mentenanță. Această structură permite adăugarea ușoară a noilor funcționalități și modificarea componentelor individuale fără a afecta întregul sistem. Testarea izolată a modulelor facilitează asigurarea calității, în timp ce designul modular permite extinderea funcționalităților existente pentru a răspunde noilor cerințe.

Capitolul 4

Implementare

4.1 Gestionarea utilizatorilor și a setărilor

Pentru a asigura securitatea și personalizarea experienței de utilizare, aplicația integrează un mecanism de autentificare locală. Parolele utilizatorilor sunt stocate în format criptat (hash) utilizând biblioteca `bcrypt` [5], recunoscută pentru rezistența sa la atacuri. Această abordare elimină riscurile asociate cu stocarea parolelor în clar și respectă bunele practici în materie de securitate a aplicațiilor desktop.

Pe lângă autentificare, aplicația permite salvarea unor setări specifice fiecărui utilizator, într-o bază de date locală de tip SQLite. Aceste setări pot include, printre altele, alegerea modului de repartizare (standard sau satisfacție mutuală), activarea funcționalităților avansate precum sistemul AI de recomandare, includerea coloanelor de detaliu pentru debugging, alegerea formatului de export sau stabilirea priorității între punctajele sociale și de performanță.

Un serviciu dedicat gestionează toate aceste preferințe, iar la inițializarea aplicației sau înaintea rulării algoritmului, ele sunt încărcate automat. Interfața grafică oferă acces facil la meniul de configurare, astfel încât utilizatorii pot modifica aceste opțiuni în mod intuitiv, fără a necesita intervenții tehnice directe asupra codului sursă.

4.2 Structura generală și fluxul de date

Aplicația este organizată în jurul unui sistem modular, în care fiecare componentă gestionează o parte distinctă a procesului de repartizare. Această modularitate oferă flexibilitate în dezvoltare și facilitează extinderea aplicației în viitor. Structura fișierelor este creată automat într-un spațiu dedicat fiecărui utilizator, în funcție de sistemul de operare: în `AppData` (pe Windows) sau în directorul `~/.fmi_accommodation` (pe Linux/macOS). Această structură include directoare pentru fișierele de intrare, fișierele procesate, rezultatele finale, fișierele generate și opțional fișiere de backup.

Pentru a garanta consistența datelor, aplicația implementează un sistem de monitorizare a fișierelor bazat pe hash-uri MD5, generate cu ajutorul bibliotecii `hashlib`. La fiecare modificare a unui fișier, aplicația recalculează hash-ul corespunzător și îl compară cu versiunea anterioară, asigurând astfel declanșarea proceselor de reîncărcare doar în caz de modificări reale, nu și în cazul deschiderilor accidentale sau al salvărilor fără conținut nou.

Pentru migrarea fișierelor între locații personalizate, se utilizează funcționalitățile cross-platform ale bibliotecilor `os` și `shutil`. Procesul de mutare implică nu doar copierea fișierelor, ci și reconstrucția structurii interne a directorului și actualizarea corespunzătoare a referințelor în baza de date locală.

Evenimentele de modificare a fișierelor sunt monitorizate cu ajutorul pachetului din python, `watchdog`, care permite o observare eficientă și în timp real a schimbărilor din sistemul de fișiere, indiferent de platforma pe care rulează aplicația. Această abordare reactivează serviciile relevante doar în momentul în care un fișier-cheie este actualizat, optimizând astfel consumul de resurse.

4.3 Gestionarea studenților

Datele despre studenți sunt extrase în principal din fișierul generat de formularul de înscriere online. Acest fișier este adesea supus unor erori de completare multiplă, motiv pentru care aplicația implementează un algoritm de deduplicare. Acest algoritm identifică înregistrările redundante folosind o combinație de câmpuri unice (CNP, număr matricol) și selectează automat o singură variantă validă pentru fiecare student. Restul sunt exportate într-un fișier dedicat de tip „duplicat”, care servește ca referință pentru verificare ulterioară.

Ulterior, aplicația segmentează automat studenții pe ani de studiu și specializări, generând câte o foaie separată pentru fiecare grup relevant. Această etapă este importantă pentru pregătirea algoritmului de distribuție proporțională a locurilor în cămine, care se bazează pe aceste categorii.

Un alt fișier esențial este exportul din sistemul UMS, care conține situațiile academice ale studenților. Aplicația extrage de aici atât punctajul de performanță, cât și punctajul social, atunci când este cazul. În cazul studenților înregistrați pe criterii sociale, aplicația oferă opțiunea de a lua în calcul valoarea maximă dintre cele două punctaje, pentru a nu dezavantaja candidații care, deși beneficiază de criterii sociale, au și performanțe academice remarcabile.

Rezultatul acestei etape este un fișier intermediar validat și sortat, care poate fi exportat în format anonimizat pentru publicarea online, conform reglementărilor GDPR. Această funcționalitate reduce semnificativ timpul de prelucrare al comisiei, un proces care anterior putea dura o zi întreagă este acum completat în mai puțin de un minut.

4.4 Gestionarea căminelor

Datele referitoare la cămine sunt furnizate anual de Direcția de Cămine sub forma unui fișier Excel, care conține informații despre denumirea fiecărui cămin, camerele disponibile și numărul de locuri corespunzătoare. Aplicația parsează acest fișier printr-un modul dedicat care extrage și structurează automat informațiile într-un format uniform, compatibil cu restul sistemului.

După importul inițial, aplicația realizează o mapare a capacităților disponibile în funcție de anul de studiu și specializarea studentului, conform cerințelor impuse de metodologia de repartizare. Pentru acest scop, este utilizat un algoritm de distribuție proporțională care analizează cererile primite, identifică cerințele pe categorii și calculează numărul de locuri alocat fiecărui segment. Algoritmul asigură că suma tuturor locurilor repartizate nu depășește capacitatea totală și ajustează automat alocările pentru a menține proporționalitatea.

Separat de procesul automat de distribuție, aplicația generează și un fișier sumariator al situației căminelor. Acest fișier este conceput pentru afișare publică și oferă o reprezentare clară și structurată a tuturor căminelor, a camerelor aferente și a capacității disponibile. Formatul este optimizat pentru lizibilitate: celulele aferente camerelor sunt grupate și îmbinate, bordurile sunt aplicate consistent, iar titlurile sunt stilizate pentru o evidențiere vizuală corectă. Aceste operațiuni de formatare sunt realizate folosind biblioteca `openpyxl`, care permite controlul detaliat al stilurilor și poziționării elementelor într-un fișier Excel.

Un alt aspect important este posibilitatea introducerii manuale a unor alocări speciale înaintea rulării algoritmului automat. Comisia de cazare poate plasa manual anumiți studenți în camere specifice. Aplicația detectează aceste cazuri, blochează camerele ocupate și exclude studenții respectivi din procesarea ulterioară, astfel încât să fie menținută coerența întregii alocări.

4.5 Algoritmul de repartizare

Algoritmul principal de repartizare este componenta centrală a aplicației. Acesta are două moduri distincte de funcționare, selectabile în interfața aplicației, precum și o etapă suplimentară de alocare de completare, care se activează automat pentru studenții care nu au putut fi repartizați în mod standard.

4.5.1 Modul standard (ordonare descrescătoare după punctaj)

În modul standard, algoritmul sortează studenții în ordine descrescătoare după punctajul general, cu posibilitatea aplicării priorității pentru cazuri sociale în funcție de setările active. Alocarea fiecărui student se face parcurgând lista preferințelor exprimate (cămin

dorit, colegi de cameră, tipul camerei). În cazul în care preferințele nu pot fi respectate complet, algoritmul face compromisuri controlate și notează aceste decizii în coloanele explicative ale fișierului final.

Pentru a permite căutări rapide și actualizări eficiente, este utilizată o structură internă bazată pe dicționare și seturi hash care stochează disponibilitatea în timp real pentru fiecare cameră, segmentate după an și specializare. În plus, se menține o evidență separată a constrângerilor de gen, astfel încât nicio alocare invalidă să nu poată fi efectuată.

4.5.2 Modul de satisfacție mutuală (bipartite matching)

Acest mod avansat este conceput pentru a răspunde nevoii studenților de a fi plasați împreună cu colegii preferați. Pentru aceasta, aplicația construiește un graf neorientat în care nodurile reprezintă studenți, iar muchiile marchează preferințele mutuale. Graf-ul este procesat cu ajutorul bibliotecii **NetworkX** [1], care oferă implementări eficiente pentru probleme de matching bipartit optim.

În această etapă, algoritmul identifică cele mai bune perechi (sau grupuri) posibile, care pot fi plasate în aceeași cameră, respectând constrângerile impuse de gen, capacitate și preferințe de cămin. Optimizarea se face folosind un algoritm greedy extins, care maximizează numărul de preferințe mutuale respectate, fără a perturba echilibrul general al alocărilor.

4.5.3 Etapa de completare (plasare forțată)

La finalul procesului, sistemul verifică lista de studenți care nu au fost plasați, din motive precum lipsa locurilor în căminele preferate sau imposibilitatea de a respecta simultan toate constrângerile. Pentru aceștia, se declanșează o rundă de alocare forțată, în care sunt căutate camere disponibile, ignorând preferințele neesențiale. Obiectivul acestei etape este de a asigura că niciun student eligibil nu rămâne nerepartizat. Aplicația documentează separat aceste alocări, oferind explicații detaliate pentru fiecare caz.

4.6 Sistemul de recomandare AI

În sprijinul studenților care nu exprimă preferințe clare pentru un cămin anume, aplicația oferă o funcționalitate de recomandare automată bazată pe procesarea limbajului natural. Acest sistem utilizează modelul pre-antrenat **all-MiniLM-L6-v2** furnizat de biblioteca **sentence-transformers**, capabil să transforme propoziții în vectori semantici (embeddings).

La inițializarea aplicației, descrierile tuturor căminelor (care includ zona geografică, facilități, atmosferă, timp de deplasare până la facultate etc.) sunt prelucrate și convertite în embeddings. Atunci când un student completează un câmp de text liber cu informații

generale (ex: „cămin aproape de metrou, liniștit”), aplicația transformă această descriere într-un vector semantic și calculează similaritatea cosinus între acesta și vectorii căminelor.

Căminul cu cea mai mare similaritate este selectat ca fiind cel mai potrivit. Dacă acesta nu mai are locuri disponibile, aplicația revine la lista preferințelor explicite sau la algoritmul standard, în funcție de opțiunile active. Sistemul de recomandare este complet integrat cu restul aplicației și poate fi activat sau dezactivat din setările generale.

4.7 Exportul și prezentarea rezultatelor

După finalizarea tuturor etapelor de procesare, aplicația generează automat fișierele de ieșire necesare, în formate standardizate. Se produc două tipuri principale de fișiere:

- **Fișierul complet** - include toate informațiile despre fiecare student, preferințele exprimate, punctajele calculate, căminul și camera repartizată, precum și coloane explicative privind deciziile de alocare (motivele pentru nerespectarea preferințelor, sursa punctajului, sursa locului etc.).
- **Fișierul public** - este generat în mod automat într-o variantă compatibilă cu cerințele GDPR, prin anonimizarea sau eliminarea informațiilor sensibile. Acesta este destinat afișării pe site-ul facultății.

Fișierele sunt salvate în format Excel, cu păstrarea formatarei originale și cu îmbunătățiri vizuale (borduri, evidențieri, grupări de camere). În funcție de opțiunile alese de utilizator, se pot genera și fișiere de diagnostic, utile pentru depanare sau pentru analiza deciziilor automate luate de sistem.

Această automatizare completă a exportului reduce semnificativ efortul administrativ, asigurând în același timp conformitate cu cerințele instituționale.

Capitolul 5

Manual de utilizare

5.1 Prezentarea generală a interfeței

Aplicația **FMI Accommodation** oferă o interfață grafică intuitivă, structurată în concordanță cu etapele procesului de prelucrare și repartizare a cererilor de cazare. Fiecare componentă funcțională este asociată unui tab distinct, iar toate acțiunile sunt reflectate în timp real prin fișierele afișate în interfață. Aplicația a fost dezvoltată utilizând biblioteca `customtkinter`, asigurând o estetică modernă și o interacțiune facilă pentru utilizator.

Interfața principală este împărțită în patru regiuni funcționale. În partea stângă se regăsește un panou lateral dedicat încărcării fișierelor esențiale: formularul completat de studenți și fișierul exportat din UMS. Tot aici se află secțiunea de sesiune, unde utilizatorul activ este afișat și poate accesa butonul de logout sau meniul de setări pentru modificarea folderului principal de lucru. Setările de criptare pot fi activate tot din acest meniu, acestea afectând modul în care sunt gestionate datele sensibile pe disc.

Centrul aplicației este rezervat tabului de acțiuni, care include toate operațiunile aplicabile asupra datelor studenților. Aceste operațiuni sunt împărțite în taburi dedicate: eliminarea duplicatelor, separarea pe ani și specializări, asocierea punctajelor și sortarea în vederea repartizării.

Sub această zonă se află secțiunea de configurare a datelor despre cămine. Utilizatorul poate introduce fișierul cu locurile disponibile, poate genera repartizarea proporțională pe categorii și poate lansa procesul final de alocare folosind algoritmul integrat.

Pe partea dreaptă a ecranului, tabul de statistici sintetizează informațiile colectate din întreg procesul, oferind vizualizări textuale și grafice pentru analiza situației curente.

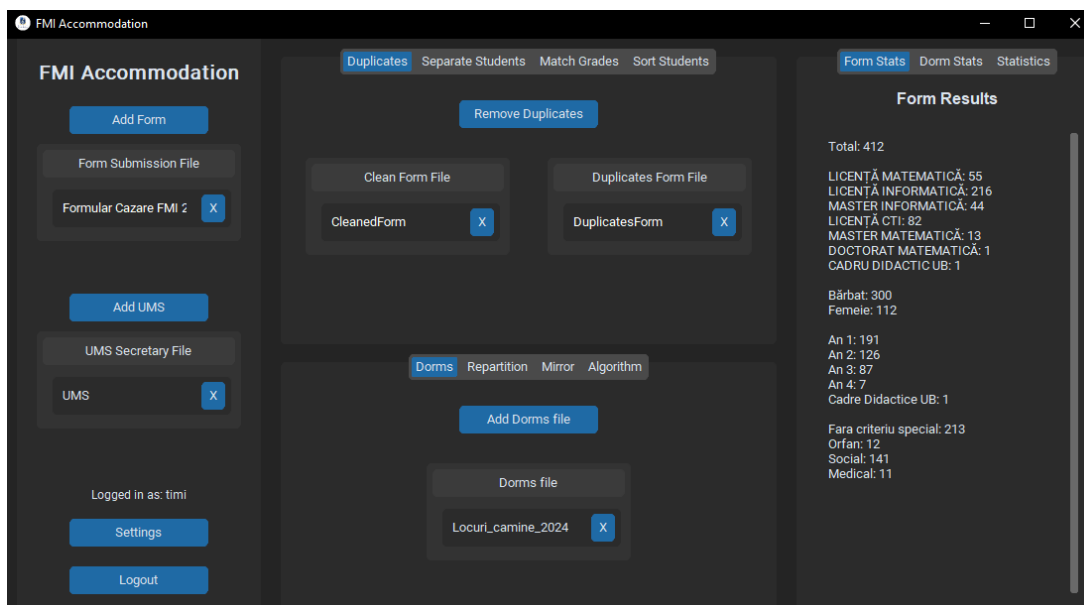


Figura 5.1: Interfața principală a aplicației — zona de acțiuni și afișare date

5.2 Procesarea datelor studenților

Zona de acțiuni (ActionsTab) acoperă pașii principali din prelucrarea datelor studenților. În primul tab, sistemul elimină automat înregistrările duplicate, identificând intrările multiple pe baza CNP-ului și numărului matricol. Se generează două fișiere: unul curățat, care va fi folosit mai departe în aplicație, și unul cu duplicatele eliminate, pentru transparență.

După curățare, datele sunt organizate în funcție de an și specializare, generând foi separate pentru fiecare categorie. Urmează asocierea punctajelor provenite din UMS, iar sistemul permite alegerea metodei de prioritizare: fie se utilizează punctajul social, fie cel de performanță, fie maximul dintre cele două, în funcție de opțiunea selectată de utilizator. La finalul acestei etape, studenții sunt sortați în ordine descrescătoare după punctajul rezultat, iar fișierele sunt actualizate automat pentru afișare și export.

5.3 Gestionarea resurselor și algoritmul de repartizare

Zona inferioară a aplicației este rezervată gestionării resurselor și lansării repartizării. Primul pas constă în introducerea fișierului transmis de Direcția de Cămine, care conține informații despre camerele și locurile disponibile în fiecare cămin. Sistemul parsează aceste date și le salvează într-o structură compatibilă cu pașii următori.

Repartizarea locurilor pe ani și specializări este realizată automat de aplicație, conform

metodologiei existente, utilizând un algoritm de distribuție proporțională. Datele sunt grupate și prezentate într-un fișier intermediar, ușor de validat de către comisie.

Ulterior, aplicația permite generarea unui fișier tabelar de tip „oglindă”, ce reflectă distribuția spațiilor de cazare și este pregătit pentru afișare publică. După această etapă, utilizatorul poate rula algoritmul final de repartizare, selectând opțiunile de personalizare dintr-o fereastră dedicată.

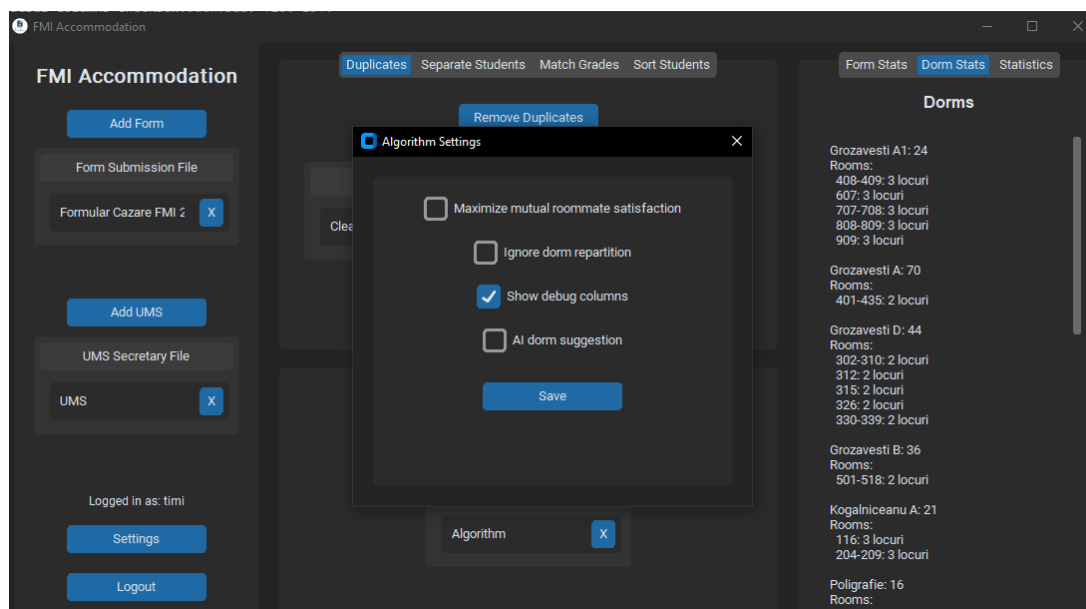


Figura 5.2: Fereastra de configurare a algoritmului de repartizare

Opțiunile disponibile includ activarea modului de satisfacție mutuală între colegii de cameră (utilizând un algoritm de matching bipartit), ignorarea completă a repartizării inițiale pe cămine (lăsând sistemul să aloce în funcție de disponibilitate), afișarea coloanelor suplimentare de analiză (debug), precum și activarea sistemului AI de sugestie pentru studenții care nu și-au exprimat preferințele.

5.4 Vizualizarea rezultatelor și statisticilor

În partea dreaptă a aplicației, secțiunea de statistici oferă o privire de ansamblu asupra datelor prelucrate. Aceasta este împărțită în trei taburi: primul prezintă distribuția studenților pe programe, ani, gen și alte criterii; al doilea descrie capacitatea fiecărui cămin, afișând lista camerelor disponibile; iar al treilea oferă o reprezentare grafică a datelor, facilitând înțelegerea rapidă a tendințelor și discrepanțelor.

Aceste instrumente vizuale contribuie semnificativ la validarea corectitudinii procesului de repartizare și oferă comisiei o unealtă suplimentară de analiză și raportare.

Capitolul 6

Concluzii

6.1 Rezumatul contribuțiilor

Lucrarea de față a avut ca scop dezvoltarea unei aplicații desktop care să automatizeze și să optimizeze procesul de repartizare a locurilor în căminele studențești, în conformitate cu metodologia aplicată în cadrul Facultății de Matematică și Informatică. Plecând de la o problemă practică observată direct în activitatea comisiei de cazare, proiectul a urmărit să ofere o soluție tehnologică eficientă, flexibilă și ușor de utilizat de către personalul administrativ.

Aplicația realizată integrează toate etapele esențiale ale procesului: de la încărcarea și curățarea datelor, asocierea punctajelor și sortarea studenților, până la împărțirea proporțională a locurilor și aplicarea algoritmilor de repartizare. Fișierele generate respectă cerințele instituționale și pot fi utilizate direct în activitatea administrativă, reducând semnificativ timpul de lucru și riscul de erori umane.

Pe lângă modul standard de repartizare, a fost inclus un algoritm alternativ bazat pe matching bipartit, care maximizează satisfacția mutuală a studenților ce doresc să fie colegi de cameră. De asemenea, aplicația dispune de un modul AI capabil să sugereze automat un cămin pe baza descrierilor libere furnizate de studenți, utilizând tehnici moderne de procesare a limbajului natural.

6.2 Limitări și provocări întâmpinate

Dezvoltarea unei aplicații care implică prelucrarea unui volum mare de date neuniforme a adus o serie de provocări, în special legate de validarea corectă a fișierelor de intrare și asigurarea compatibilității între diferite surse (formularul de înscriere, datele din UMS, fișierul cu locuri disponibile).

O altă dificultate a fost reprezentată de nevoia de flexibilitate ridicată a algoritmului, pentru a permite intervenții manuale ale comisiei în etapele sensibile ale procesului.

Această cerință a fost rezolvată prin implementarea unui sistem modular de setări și opțiuni personalizabile, care pot influența direct comportamentul algoritmului fără a necesita modificări ale codului.

Deși aplicația acoperă majoritatea scenariilor uzuale, pot exista cazuri particulare care să necesite ajustări punctuale sau completări ulterioare, în funcție de eventualele modificări în metodologia facultății.

6.3 Direcții de dezvoltare viitoare

Aplicația implementată reprezintă o bază solidă pentru extindere și rafinare. Printre direcțiile posibile de dezvoltare viitoare se numără:

- Integrarea aplicației cu platforma UMS prin API, pentru importul automat al datelor despre studenți.
- Adăugarea unei interfețe pentru generarea și trimiterea notificărilor automate către studenți, după finalizarea procesului de repartizare.
- Dezvoltarea unui modul de audit și istoric al deciziilor luate în aplicație, util în cazul unor contestații sau modificări ulterioare.
- Extinderea sistemului de sugestii AI prin antrenarea unui model personalizat pe preferințele exprimate în anii anteriori.
- Adaptarea aplicației la nivelul întregii universități, cu opțiuni de configurare per facultate.

6.4 Reflecție personală

Această lucrare a fost mai mult decât un proiect academic — a fost o ocazie de a contribui direct la îmbunătățirea unui proces administrativ concret și important din cadrul facultății. Am avut șansa să lucrez la o problemă reală, cu cerințe și constrângeri specifice, și să văd impactul practic al unei soluții software bine concepute.

Procesul de dezvoltare m-a ajutat să înțeleg mai bine aspectele aplicate ale programării orientate pe date, gestionarea structurilor complexe de fișiere și importanța gândirii algoritmice în rezolvarea unor sarcini organizaționale. De asemenea, am învățat cum să proiectez o interfață prietenoasă, menținând în același timp o arhitectură internă clară și scalabilă.

Consider că această experiență a reprezentat un pas important în formarea mea profesională și sunt convins că aplicația va putea fi utilizată și îmbunătățită în anii următori, sprijinind comisia de cazare și oferind un exemplu de digitalizare eficientă în mediul universitar.

Bibliografie

- [1] Aric Hagberg, Pieter Swart și Daniel S. Chult, *Exploring Network Structure, Dynamics, and Function using NetworkX*, 2008.
- [2] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Mattheus Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke și Travis E. Oliphant, „Array programming with NumPy”, în *Nature* 585 (2020), pp. 357–362, DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [3] Wes McKinney, „Data Structures for Statistical Computing in Python”, în (2010), pp. 56–61, DOI: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
- [4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai și Soumith Chintala, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, 2019, URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [5] Niels Provos și David Mazières, *A Future-Adaptable Password Scheme*, 1999, URL: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>.
- [6] Nils Reimers și Iryna Gurevych, „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, în (2019), pp. 3982–3992, URL: <https://arxiv.org/abs/1908.10084>.
- [7] Tom Schimansky, *CustomTkinter: A modern and customizable python UI-library based on Tkinter*, 2022, URL: <https://github.com/TomSchimansky/CustomTkinter>.